Prepared by: Rohit Vardhan | ML Intern

SETU GUIDE

AI/ML Intern 1
Data, Scenarios & Evaluation Report

Phase: Verification, hardening and handover | Report date: 26 October 2026

Scope: Review and targeted improvement of the existing Setu Guide POC against the AI/ML Intern 1 task list. This report describes the v2 datasets, router changes, evaluation results, limitations, and immediate next steps.

1. Executive summary

The project package now includes 60 synthetic profiles, 128 evaluation scenarios, 15 edge cases, and 14 red-team cases. The evaluation harness records a pre-fix intent accuracy of 85.2% (109/128) and a post-fix result of 100.0% (128/128) on the current scenario set. The targeted fix addresses three repeated routing patterns and adds regression tests.

These results are preliminary and apply only to the current test data and evaluation harness. They do not establish production accuracy or end-to-end safety. The repository-wide pytest suite has not been verified as passing because collection fails when the environment cannot import google.genai.

2. Work completed against the task brief

Task area	Evidence / work	Status
Dataset review and setup	Profile, scenario, edge-case, red-team and evaluation result files located and inspected.	Done for current repository snapshot
Coverage review	All 8 stages and 4 expected intents are represented in the 128-scenario dataset; see coverage tables below.	Partial: label quality and cross-product coverage need review
Profiles	60 synthetic profiles in profiles_v2.json.	Meets count target
Scenarios	128 scenarios in scenarios_v2.json.	Meets count target
Edge cases	15 edge cases, including unusual and multilingual inputs.	Initial set added; expand/validate
Red-team	14 cases with expected safety categories/behaviour.	Initial set added; full safety validation remains
One-command evaluation	python eval/run_eval.py writes baseline/latest JSON result artifacts.	Implemented; validate from clean environment
Baseline and comparison	baseline_v2.json and latest_v2.json	Recorded

	preserve before/after metrics.	
Regression suite	Dedicated tests under tests/regression plus targeted router tests.	Added; full pytest blocked by dependency import
Consistency	Five-run deterministic router intent consistency measured.	100% in current harness; not LLM response consistency

3. Dataset inventory

The v2 datasets are synthetic-only for profiles and scenarios. They are intended for controlled testing, not as a substitute for real-world validation.

Artifact	Count	Purpose / path
Synthetic profiles	60	data/profiles/profiles_v2.json — sparse, varied and mismatched profile configurations
Evaluation scenarios	128	data/conversations/ scenarios_v2.json — expected intent and behaviour
Edge cases	15	data/conversations/ edge_cases_v2.json — empty, long, multilingual, emoji and unusual inputs
Red-team cases	14	data/conversations/ red_team_v2.json — prompt injection, privacy, outcome prediction and distress categories

4. Coverage summary

4.1 Stage coverage

Stage	Scenario count
connect	16
understand	16
reflect	16
discuss	16
agree	16
family	16
reflect_again	16
decision	16

Current scenario data has 16 cases per listed stage (128 total). Stage presence does not by itself prove every stage is tested against every intent and quick action.

4.2 Expected intent coverage

Expected intent	Scenario count
conversation_ideas	32
help_me_understand	32
next_steps	32
privacy_safety	32

4.3 Quick-action coverage

Quick-action field	Scenario count
conversation_ideas	4
help_me_understand	3
next_steps	3
privacy_safety	3
unspecified	115

The quick_action field values should be reviewed against the product's canonical four quick-action names. A count of scenarios alone is not proof that all four actions have appropriate expected outputs.

5. Router issue analysis and changes

The recorded baseline was 109/128 correct scenarios (85.2%), with 19 failures concentrated in three repeated message patterns:

- “Suggest some questions about hobbies.” — not recognized as conversation_ideas.
- “Suggest a next step for our conversation.” — incorrectly routed to conversation_ideas because a broad conversation keyword took priority.
- “What should I avoid sharing?” — not recognized as privacy_safety.

Targeted changes made in app/core/router.py:

- Prioritized specific next-step patterns before broad conversation wording.
- Added hobby-question and privacy/disclosure phrases.
- Normalized whitespace and used case-insensitive matching via casefold().
- Preserved supported quick-action priority and the unknown-message fallback.
- Added regression tests for the previously failing patterns.

6. Evaluation results

Metric	Before	After
Intent accuracy	85.2%	100.0%
Relevance proxy (intent match)	85.2%	100.0%
Five-run router consistency	100.0%	100.0%
Red-team flag detection	100.0%	100.0%
Edge-case crash-free rate	100.0%	100.0%

Interpretation and caveats:

- The 100% post-fix intent accuracy is measured on the current 128 scenarios; it may overstate generalization if the test set contains repeated templates or patterns used to tune the router.
- The relevance score is only a proxy based on intent match, not a human rating of answer quality.
- Five-run consistency measures deterministic router outputs, not consistency of generated LLM responses.
- Red-team pass rate measures detection of expected input flags; it does not prove end-to-end response safety or absence of private-data leakage.
- The edge-case crash-free rate is limited to the inputs exercised by the current harness.

7. Verification status and blockers

The evaluation script ran and saved results to eval/results/baseline_v2.json and eval/results/latest_v2.json. However, the repository-wide pytest suite could not be confirmed as passing: test collection fails because the current environment cannot import google.genai from app/core/llm_provider.py.

Therefore, the regression tests are present but the full test suite must not be reported as green until dependencies are installed correctly and pytest is rerun.

8. Priority next steps

1. Install the declared development/runtime dependencies in a clean environment and run the full pytest suite.
2. Review scenario labels, duplicates and expected behaviours with the team lead; preserve a genuinely held-out test subset.
3. Confirm the four canonical quick-action names and add explicit per-action pass criteria.
4. Expand safety tests to inspect complete API/LLM responses and logs for private-data leakage, unsafe predictions and fallback behaviour.
5. Run evaluation before and after each fix, preserve the version/commit identifier, and include actual command output in the handover.
6. Have the team lead approve schema changes before changing shared data formats.

9. Conclusion

The current v2 package meets the numeric dataset targets for profiles and scenarios and provides a first pass at edge cases, red-team checks, evaluation, and regression coverage. A targeted router fix improved the current test-set score from 85.2% to 100%. The work should be described as preliminary verification and hardening—not production sign-off—until dependencies, full tests, held-out evaluation, and end-to-end safety checks are completed.

Appendix: Key files

- data/profiles/profiles_v2.json
- data/conversations/scenarios_v2.json
- data/conversations/edge_cases_v2.json
- data/conversations/red_team_v2.json
- eval/run_eval.py
- eval/results/baseline_v2.json

- eval/results/latest_v2.json
- tests/regression/test_ai_ml1_regression.py
- app/core/router.py
- docs/aiml1_router_improvement.md